Session 04: Git & GitHub

DevOps Fundamentals

RIKKEI Academy®
where the dream begins

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NỘI DUNG

Lesson 01: Khởi tạo và thao tác cơ bản với Git (Local)

Lesson 02: Quản lý nhánh, xử lý xung đột với Git

Lesson 03: Làm việc từ xa với GitHub, quy trình Teamwork

Lesson 04: Kỹ thuật Git nâng cao, quản lý lịch sử Git

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

1

Khởi tạo và thao tác cơ bản với Git (Local)

Quản lý phiên bản bằng tay

Cách làm quen thuộc trước khi biết Git — và ba vấn đề nó không bao giờ giải được

TERMINAL — THƯ MỤC DỰ ÁN

```
$ ls -1 ~/projects/
taskapi/
taskapi-final/
taskapi-final-v2/
taskapi-final-v2-sua-lai/
taskapi-BACKUP-27-08/
taskapi-ban-cua-Nam/

# Bản nào đang chạy được trên production?
# Nam đã sửa đúng những gì trong bản của Nam?
# Hôm qua nó còn chạy — hôm qua là bản nào?
```

Không biết ai sửa gì

Không có tác giả, không có thời điểm và không có lý do cho từng thay đổi

Không quay lại được

Sửa hỏng một file là mất luôn phiên bản chạy được ngay trước đó

Không làm song song

Hai người sửa cùng một file thì một người phải chờ hoặc ghi đè lên nhau

Git giải quyết cả ba: mỗi thay đổi có tác giả và thời điểm, mọi phiên bản cũ đều quay lại được, và nhiều người làm song song trên cùng một dự án.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Ba vùng dữ liệu của Git

Mọi lệnh Git cơ bản chỉ là việc di chuyển thay đổi giữa ba vùng này



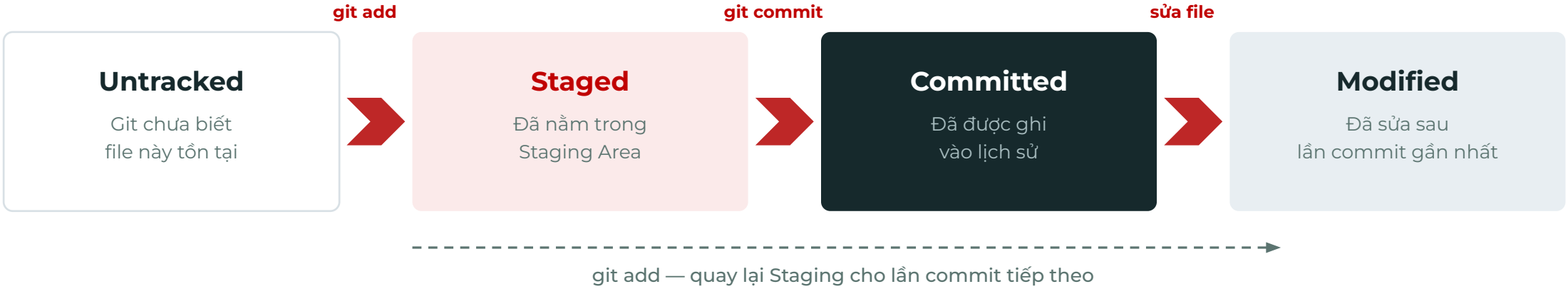
1	**Working Directory**	Nơi bạn thật sự gõ code. Git chưa quan tâm gì cho tới khi bạn đưa thay đổi sang vùng kế tiếp
2	**Staging Area**	Bản nháp của lần commit tới. Cho phép chọn đúng những thay đổi muốn ghi lại, bỏ qua phần còn dở
3	**Local Repository**	Toàn bộ lịch sử nằm trong thư mục .git ngay trên máy bạn — không cần mạng vẫn xem lại được

Ghi nhớ: toàn bộ lịch sử nằm trong thư mục .git. Xoá .git là mất repo chứ không mất code — code vẫn còn nguyên trong Working Directory.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Vòng đời của một file trong Git

Bốn trạng thái và những lệnh làm file chuyển từ trạng thái này sang trạng thái khác



git status — ba nhóm luôn xuất hiện

```
Changes to be committed:      # đang ở Staging
  modified:   src/user.service.js
Changes not staged for commit:  # mới sửa
  modified:   src/index.js
Untracked files:              # Git chưa biết
  .env.local
```

Đọc git status trước mọi thao tác
Nó cho biết chính xác file nào đang ở vùng nào, không phải đoán

.gitignore chặn ngay từ đầu
node_modules, .env, file build không bao giờ nên vào lịch sử

Staging cho phép commit chọn lọc
Sửa ba việc trong một buổi vẫn tách được thành ba commit riêng

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Sáu lệnh dùng mỗi ngày

Chín mươi phần trăm công việc hằng ngày với Git chỉ nằm trong sáu lệnh này

Lệnh	Làm gì	Khi nào dùng
git init	Tạo thư mục .git, biến folder thành repository	Bắt đầu một dự án mới
git status	Liệt kê file đang nằm ở vùng nào	Trước mỗi lần add và commit
git add <file>	Đưa thay đổi từ Working Directory vào Staging	Khi hoàn thiện một phần việc
git commit -m	Ghi ảnh chụp của Staging vào lịch sử	Khi có một thay đổi trọn vẹn
git log --oneline	Xem lịch sử commit dạng rút gọn	Khi cần tìm lại một thay đổi cũ
git diff	So sánh nội dung giữa hai vùng	Soát lại trước khi add và commit

Cẩn thận với git add . : nó thêm mọi thứ, kể cả file rác và file bí mật. Viết .gitignore ngay từ commit đầu tiên và luôn đọc git status trước khi commit.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Một commit tốt trông như thế nào?

Lịch sử là tài liệu của dự án — người đọc nó nhiều nhất chính là bạn của sáu tháng sau

```
git show --stat HEAD

commit 9f3c1ab7e2d54c8b1f06a3   # SHA-1
Author: Nguyen <nguyen@rikkei.edu.vn>
Date:   Wed Aug 12 09:14:22 2026 +0700

    feat(auth): thêm refresh token cho login

    Access token hết hạn sau 15 phút khiến người
    dùng bị đăng xuất giữa chừng. Refresh token
    giữ phiên 7 ngày mà không lưu mật khẩu.

 src/auth/token.service.js   | 42 ++++++
 src/auth/auth.controller.js |  8 +++-
$ git log --oneline -2
9f3c1ab feat(auth): thêm refresh token
```

1. **Một commit, một việc**
   Commit nhỏ thì dễ review, dễ revert và dễ tìm ra nguyên nhân khi lỗi

2. **Dòng đầu dưới 50 ký tự**
   Viết ở thể mệnh lệnh: "thêm", "sửa", "xoá" — không viết "đã thêm"

3. **Tiền tố theo loại thay đổi**
   feat · fix · refactor · docs · test · chore giúp lọc lịch sử rất nhanh

4. **Phần thân trả lời "vì sao"**
   Diff đã nói cái gì thay đổi; message phải nói vì sao cần thay đổi

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

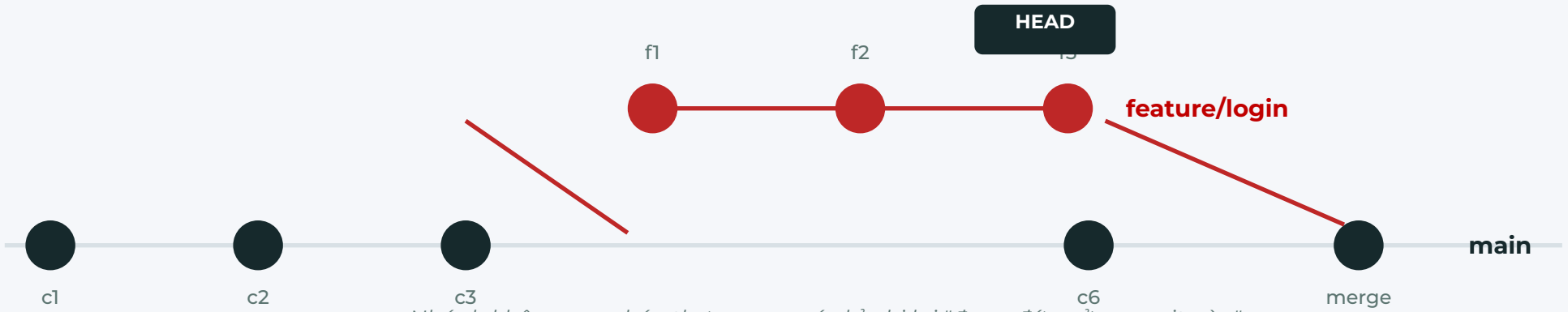
RIKKEI Academy®
where the dream begins

2

Quản lý nhánh, xử lý xung đột với Git

Nhánh chỉ là một con trỏ tới commit

Tạo nhánh trong Git gần như miễn phí — đó là lý do quy trình làm việc xoay quanh nhánh



Nhánh không sao chép thư mục — nó chỉ ghi lại "đang đứng ở commit nào"

```
$ git switch -c feature/login   # tạo và chuyển
$ git branch                    # liệt kê nhánh
  main
* feature/login                 # dấu * là HEAD
$ git switch main               # quay về main
$ git branch -d feature/login   # xoá sau merge
```

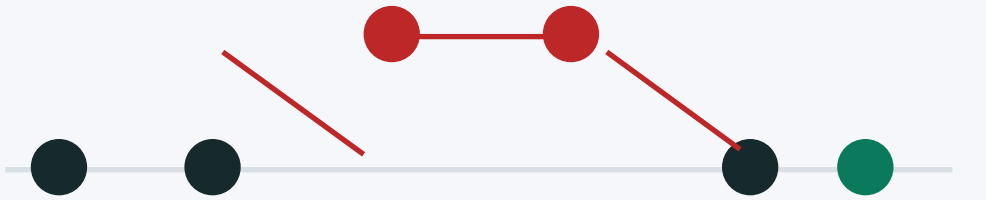
Mỗi việc một nhánh	main luôn ở trạng thái chạy được; việc đang dở nằm trên nhánh riêng
Đặt tên có quy tắc	feature/... · fix/... · hotfix/... — nhìn tên là biết nhánh làm gì
Xoá nhánh sau khi merge	Nhánh cũ còn lại chỉ gây nhiễu; lịch sử đã nằm trong main rồi

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Merge hay Rebase?

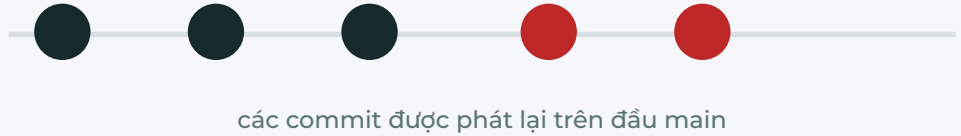
Cùng đưa công việc của nhánh vào main, nhưng để lại hai kiểu lịch sử hoàn toàn khác nhau

GIT MERGE



- Giữ nguyên lịch sử đúng như đã xảy ra
- Sinh thêm một commit hợp nhất trong đồ thị
- Đồ thị có nhiều nhánh, khó đọc khi team đông

GIT REBASE



- Lịch sử thẳng một hàng, rất dễ đọc lại
- Viết lại hash của các commit được phát lại
- Không rebase nhánh đã push và chia sẻ chung

Quy ước phổ biến: rebase nhánh cá nhân của mình để dọn lịch sử trước khi mở Pull Request, và dùng merge khi đưa Pull Request vào main.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Xung đột trông như thế nào?

Git tự hợp nhất được hầu hết trường hợp — nó chỉ dừng lại khi hai nhánh sửa đúng cùng một đoạn

src/config.js — sau khi git merge dừng lại

```
<<<<<<< HEAD              # nhánh hiện tại
const PORT = process.env.PORT || 3000;
=======                   # ranh giới
const PORT = process.env.PORT ?? 8080;
>>>>>>> feature/config    # nhánh đang gộp vào

$ git status
You have unmerged paths.
Unmerged paths:
  both modified:   src/config.js
```

Xung đột không phải lỗi

Đó là Git từ chối đoán bừa và giao quyết định lại cho con người

Chỉ xảy ra khi trùng vùng

Hai người sửa hai hàm khác nhau trong cùng một file thì Git tự gộp được

Ba khối, hai lựa chọn, một quyết định

Nhiều khi đáp án đúng không phải bên nào cả mà là một đoạn code thứ ba

Đọc dấu hiệu: phần giữa <<<<<<< và ======= là code của nhánh bạn đang đứng; phần giữa ======= và >>>>>>> là code của nhánh đang được gộp vào.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Năm bước giải quyết một xung đột

Luôn theo đúng thứ tự này — bỏ qua bước nào cũng dẫn tới việc phải làm lại từ đầu

1	**Đọc git status**	Nó liệt kê chính xác file nào đang xung đột — đừng mở bừa từng file để đoán
2	**Mở file, tìm dấu <<<<<<<**	Xác định rõ đâu là code của nhánh mình, đâu là code của nhánh kia
3	**Chọn hoặc viết lại đoạn code**	Không nhất thiết phải chọn một bên; thường phải viết lại cho đúng cả hai ý
4	**Xoá sạch mọi dấu xung đột**	Còn sót một dòng <<<<<<< là code không chạy được và lọt lên remote
5	**git add rồi git commit**	add đánh dấu file đã giải quyết xong, commit đóng lại lần hợp nhất

`git merge --abort` · `git rebase --abort` · `git checkout --ours <file>` · `git checkout --theirs <file>`

Không hoảng: git merge --abort đưa mọi thứ về đúng trạng thái trước khi merge. Không có thao tác nào là không thể quay lại.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

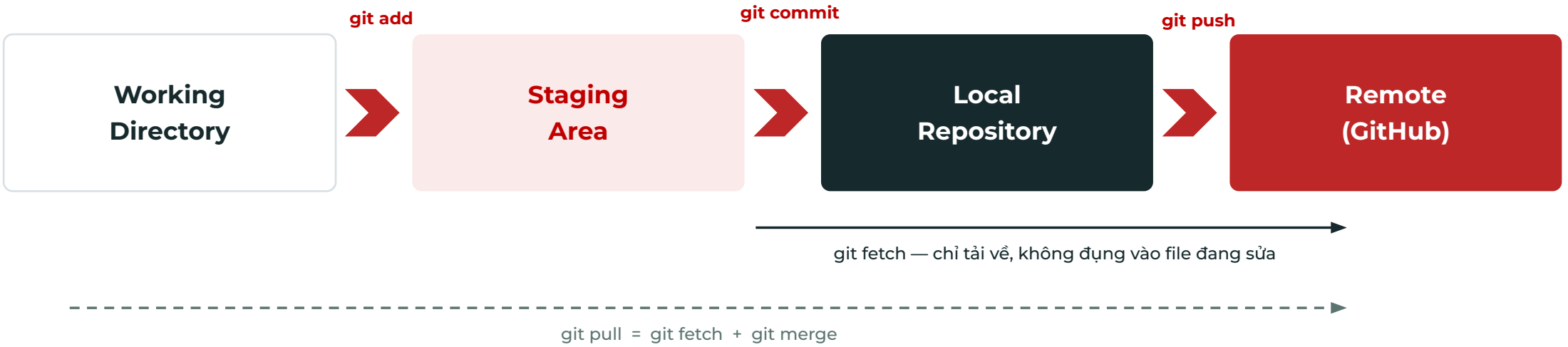
RIKKEI Academy®
where the dream begins

3

Làm việc từ xa với GitHub, quy trình Teamwork

Từ máy cá nhân lên GitHub

Thêm một vùng thứ tư vào sơ đồ cũ — và bốn lệnh để di chuyển dữ liệu giữa chúng




```
$ git remote add origin git@github.com:team/taskapi.git
$ git push -u origin main     # -u nhớ nhánh cho lần sau
$ git clone <url>             # tải toàn bộ repo về máy
$ git remote -v               # kiểm tra đang trỏ tới đâu
```

origin là tên quy ước	Chỉ là bí danh của một URL — đổi tên khác vẫn chạy bình thường
Remote không tự đồng bộ	Không push thì GitHub không biết gì về commit trên máy bạn
Clone khác download ZIP	Clone mang theo toàn bộ lịch sử và kết nối tới remote

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Bốn lệnh đồng bộ, bốn tác dụng

Nhầm lẫn giữa fetch và pull là nguyên nhân phổ biến nhất của những lần mất code

Lệnh	Hướng dữ liệu	Chạm vào vùng nào	Ghi nhớ
git clone	Remote → Máy bạn	Tạo mới toàn bộ repo	Chỉ chạy một lần khi bắt đầu tham gia dự án
git fetch	Remote → Local repo	Chỉ cập nhật origin/*	An toàn tuyệt đối, không đụng file đang sửa
git pull	Remote → Working dir	fetch rồi merge luôn	Có thể sinh xung đột ngay lập tức
git push	Local repo → Remote	Cập nhật nhánh trên GitHub	Bị từ chối nếu remote đã đi trước bạn

```
# xem trước rồi mới gộp: git fetch → git log --oneline main..origin/main → git merge origin/main
```

Khi push bị từ chối: nghĩa là có người đã push trước bạn. Hãy pull (hoặc fetch rồi rebase) chứ tuyệt đối không dùng push --force trên nhánh chung.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Vòng đời của một Pull Request

Sáu chặng lặp lại mỗi ngày trong một team — đây là nơi Git trở thành quy trình làm việc

1 **Tạo nhánh**
tách khỏi main
`git switch -c`

2 **Commit**
từng việc nhỏ
`git commit -m`

3 **Push**
đẩy nhánh lên
`git push -u`

4 **Mở PR**
base ← compare
trên GitHub

5 **Review**
sửa và push tiếp
PR tự cập nhật

6 **Merge**
và xoá nhánh
`squash / merge`

PR là đơn vị công việc

Một PR nên tương ứng với một task — dễ review, dễ revert khi cần

Nhánh vẫn sống trong lúc review

Push thêm commit vào nhánh thì PR tự cập nhật, không cần mở PR mới

Merge xong là xoá nhánh

Lịch sử đã nằm trong main; giữ lại nhánh cũ chỉ làm rối danh sách

Không bao giờ push thẳng vào main. Mọi thay đổi đi qua Pull Request, kể cả khi bạn là người duy nhất trong dự án.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Code Review từ hai phía

Review là để bảo vệ codebase và truyền kiến thức, không phải để chấm điểm con người

NGƯỜI GỬI PULL REQUEST	NGƯỜI REVIEW
Giữ PR nhỏ — dưới 400 dòng thay đổi là lý tưởng	Đọc phần mô tả trước khi nhảy vào đọc code
Tự đọc lại toàn bộ diff trước khi bấm gửi	Hỏi "vì sao chọn cách này?" thay vì ra lệnh sửa
Mô tả rõ thay đổi gì, vì sao, và kiểm thử thế nào	Phân biệt rõ "bắt buộc sửa" và "gợi ý thêm"
Chạy test và lint xong mới mở PR	Kiểm tra test có bao phủ đúng phần thay đổi không
Không gộp nhiều việc không liên quan vào một PR	Phản hồi trong vòng 24 giờ, đừng để PR nằm chờ

Một câu đáng nhớ: nhận xét về đoạn code, không nhận xét về người viết. "Hàm này có thể lỗi khi mảng rỗng" khác hẳn "bạn viết ẩu quá".

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

4

Kỹ thuật Git nâng cao, quản lý lịch sử Git

git stash — ngăn kéo tạm thời

Đang sửa dở mà phải chuyển nhánh gấp: cất tạm thay đổi thay vì commit nửa vời

```
TERMINAL

# đang sửa dở thì có bug production cần vá gấp
$ git status
  modified:  src/user.service.js

$ git stash push -m "dang sua validate"
Saved working directory and index state

$ git switch hotfix/login-500   # cây làm việc đã sạch
  ... vá xong, commit, push ...

$ git switch feature/user
$ git stash list
stash@{0}: On feature/user: dang sua validate
$ git stash pop          # lấy lại và xoá stash
```

1. **stash push**
   Cất mọi thay đổi chưa commit vào ngăn tạm và trả Working Directory về sạch sẽ

2. **stash list · stash show**
   Xem đang cất những gì và nội dung cụ thể của từng lần cất

3. **pop khác apply**
   pop lấy ra rồi xoá khỏi danh sách; apply giữ lại bản cất để còn dùng tiếp

4. **Đừng cất quá lâu**
   Stash không bao giờ được push lên remote — hỏng máy là mất trắng phần đang cất

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Hoàn tác: revert, reset hay restore?

Ba lệnh nghe giống nhau nhưng chạm vào những vùng khác nhau — chọn nhầm là mất code

Lệnh	Viết lại lịch sử	Chạm vào vùng nào	Dùng khi	Đã push?
git revert <hash>	Không — thêm commit mới	Lịch sử (thêm 1 commit đảo)	Hoàn tác commit đã công khai	AN TOÀN
git reset --soft	Có	Chỉ con trỏ nhánh	Gộp lại vài commit cuối	KHÔNG
git reset --mixed	Có	Con trỏ + Staging	Muốn add và commit lại từ đầu	KHÔNG
git reset --hard	Có	Con trỏ + Staging + file	Xoá sạch mọi thay đổi chưa lưu	KHÔNG
git restore <file>	Không	Chỉ file trong Working Dir	Bỏ phần sửa dở của một file	AN TOÀN

Nguyên tắc vàng: commit đã push lên nhánh chung thì chỉ được hoàn tác bằng revert. reset viết lại lịch sử và buộc phải force push — việc đó làm hỏng repo của cả team.

Lỡ tay reset --hard? git reflog vẫn còn giữ dấu vết trong khoảng 90 ngày.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Ba mức của git reset

Cùng lùi con trỏ nhánh, khác nhau ở chỗ Staging Area và file của bạn có bị đụng tới hay không

	Con trỏ HEAD	Staging Area	Working Directory
--soft	**Lùi lại**	**Giữ nguyên**	**Giữ nguyên**
Commit biến mất khỏi lịch sử nhưng mọi thay đổi vẫn nằm sẵn trong Staging			
--mixed	**Lùi lại**	**Bị xoá**	**Giữ nguyên**
Mặc định. Thay đổi vẫn còn trong file nhưng phải git add lại từ đầu			
--hard	**Lùi lại**	**Bị xoá**	**Bị xoá**
Xoá sạch. Mọi thay đổi chưa commit sẽ mất và không có cảnh báo			

```
git reset --soft HEAD~1  ·  git reset --mixed HEAD~2  ·  git reset --hard origin/main
```

Mẹo an toàn: trước khi reset --hard, chạy git stash một lần. Mất 2 giây và cứu bạn khỏi việc mất cả buổi làm.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

TỔNG KẾT

- Ghi lại lịch sử bằng ba vùng làm việc và commit sạch sẽ
- Tách việc trên nhánh, hợp nhất và tự giải quyết xung đột
- Cộng tác qua GitHub bằng Pull Request và Code Review
- Cứu và sắp xếp lịch sử với stash, revert, reset, cherry-pick

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN